

Exam One Review

Paul Robinson

September 21, 2026

1. Assuming all numbers are stored as integers
 - a) $(2+3*4-6 = 8)$
 - b) $(5+11/3 = 8)$
 - c) $(11 \% 3 * 4 = 8)$
 - d) $((2+1)*3-1 = 8)$
2. Create Boolean Test Conditions
 - a) `(myHeight > 2)`
 - b) `(y % 2 == 0) && (y < 10)`
 - c) `(x == 3) || (y == 3)`
 - d) `(t > 2.1) && (t < 2.3)`
3. What is the Output?
 - a) The output is "12"
4. What is the Output?
 - a) The output is "1"
5. What is the Output?
 - a) The output is "Have a good day!"
6. What is the Output?
 - a) The output is "2"
7. What is the Output?
 - a) The output is "4"
 - b) The output is then "4.5"
 - c) The output is then "4.5"
 - d) The output is then "4.5"
8. What is the Output?
 - a) The output is "6 5"
 - b) The output is then "7 7"
9. Find all errors
 - a) z is not given a type
 - b) y is not typecasted inside of the brackets
10. Write if/else code
 - a) `cout << "your score was: " << exam_score << endl;`
 - b) Inside a block:

```
int get_score(int exam_score) {  
    if (exam_score < 80) { return "C"; }  
    if (exam_score < 90) { return "B"; }
```

```
        return "A";
    }
```

11. Write loop code

a) // Yeah I know we're supposed to start from 0, but why waste a loop call

```
int index = 1;
int max;
cout << "Give a number to count odd numbers to: ";
cin >> max;
```

```
while (index <= max) {
    cout << index << endl;
    index += 2;
}
```

b) `int` max;
cout << "Give a number to count odd numbers to: ";
cin >> max;

```
for (int index = 1; index <= max; index += 2) {
    cout << index << endl;
}
```

12. Rewrite as a Switch

```
switch(rank) {
    case 1:
    case 2:
        cout << "Lower Division" << endl;
        break;
    case 3:
    case 4:
        cout << "Upper Division" << endl;
        break;
    case 5:
        cout << "Graduate Student" << endl;
        break;
    default:
        cout << "Invalid Rank" << endl;
        break;
}
```

13. True/False

- a) True
- b) False
- c) True
- d) False

14. Rewrite as a for loop

a) `for` (`int` i = 2; i <= 18; i += 3) {
 cout << "*";
}

b) The output is: *****

15. What is the output? The output is: 0
16. What is the output? The output is: (infinity) or integer overflow.
17. What is the output? The output is: 90
18. What is the output?

```
*****
*****
***
**
```

19. Write a program

```
// This Program should produce ten random integers between 1 and 100
#include <random>
#include <iostream>

using namespace std;

int get_random_number(int min, int max) {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<int> distrib(min, max);
    return distrib(gen);
}

int main() {
    for (int i = 0; i < 10; i++) {
        cout << i << ": " << get_random_number(1, 100) << endl;
    }
    return 0;
}
```

20. True or False

- a) True
- b) False

21. What is the output?

```
x = 0
y = 0
(The function should take in parameters as pointers or references rather than
variables)
```

22. What is the output?

```
d = 34
(The function should return a double, so in practice it may give warnings.
Otherwise, you need to either change it or add a type cast to the function call)
```

23. Which are legal statements? Statements **A**, **B**, and **F** are legal statements. The others either have an issue with the return type or mismatched parameters.

24. Write Functions

- a) // no return and no functionality


```
void cool_func() {
    // be pointless
    return;
}
```
- b) // double return and no functionality


```
double cool_func() {
    double d = 0.0;
    return d;
}
```
- c) // integer return and no functionality


```
int neutral_func(int foo, double bar, char baz) {
    int return_val = 0;
    return return_val;
}
```

25. Write a function

```
// gets the area of a square of whole numbers
int get_area(int base) {
    return base * base;
}
```

26. Find the errors

- a) The function should be defined before `main` (or use a forward declaration).
- b) `add_one` doesn't return anything but has a return type assigned.
- c) `int 7;` is illegal and should be replaced with `int x = 7;`.
- d) `add_one`'s call should have `x` assigned to it: `x = add_one(x);`.

27. Memory Questions

- a) It holds the sign of the number (Sign bit).
- b) If the integer is signed, the maximum positive threshold value is halved to allow for negative numbers.

28. Data Type Modifiers

- a) `unsigned long long int` allows storing only positive integers from (0) to $(2^{\{64\}}-1)$.
- b) `long long int` allows storing positive and negative integers from $(-2^{\{63\}})$ to $(2^{\{63\}}-1)$.
- c) One allows for negative numbers and one doesn't, which doubles the positive capacity spectrum.

29. Data Type Modifiers Extended B is the answer because assigning -1 to an unsigned integer triggers a predictable wrap-around behavior due to modulo arithmetic, turning it into the maximum possible limit.

30. Building Object files are larger than C++ files because they contain verbose compiled machine code, symbol lookup tables, and the fully copy-pasted contents of every expanded `#include` header file.

31. Makefiles

- a) Makefiles implement incremental builds, tracking modified file headers to only compile updated source files, saving massive amounts of build automation time.
- b) We separate the compilation and linking phases so code changes only force a local item compilation without recalculating the entire project.

32. Write a Function

```
void make_tree(int base) {  
    for(int i = 0; i < base; i++) {  
        int spaces = base - i - 1;  
        int leaves = i * 2 + 1;  
  
        for (int k = 0; k < spaces; k++) cout << " ";  
        for (int k = 0; k < leaves; k++) cout << "*";  
        cout << endl;  
    }  
  
    int trunk_spaces = base - 1;  
    for (int k = 0; k < trunk_spaces; k++) cout << " ";  
    cout << "x" << endl;  
}
```